

PackRight — Owner Action Checklist

luggageliason.com · everything the August 2026 audit still needs that could not be completed in code

Prepared 29 August 2026

Operator Oak and Main Developers LLC · 2108 N St., Sacramento, CA 95816

Repository kevynsgrin-a11y/PackRight · branch `claude/luggageliason-audit-p0-p1-xxun0z`

Cloudflare zone luggageliason.com · active, Free plan

Already done, no action needed.

- All P0 (01–08) and P1 (09–17) audit issues closed in code, merged to `main`, CI green.
- **Production database migrated.** `packright-db` now carries the full provenance schema — 8 airlines with slugs and source URLs, 13 fare families, 5 benefits, 4 TSA rules, 4 labelled assumptions, and the `change-log` table. Rehearsed against a schema replica first, which caught a defect that would have silently wiped benefit provenance.
- **Accessibility:** zero axe-core violations across 13 routes and 5 error/warning states. 24 keyboard stops, none unnamed, all with a visible focus ring. No horizontal overflow at 320/390/768/1440 px.
- **Security headers verified** as actually served, with the CSP proven not to block anything the real pages load. Service worker verified: installs, works offline, and correctly refuses to serve a cached API response.
- Legal entity, postal address, California governing law and a Do Not Track statement added to the policy pages.
- **A second, adversarial audit** across 12 dimensions found and fixed further defects, including a regression where the UI rebuilt a green “Verified” badge from a date for records the API had marked unverified, a malformed `Cache-Control` on every rate-limit response, a render crash that blanked the page on a malformed API body, a CI lint gate that could never fail, and a fabricated Southwest dimension in my own dataset.
- **That audit then completed in full** — 37 confirmed findings, 26 rejected, no P0s — and all of them are now fixed. The ones worth naming: a card waiver could cover an entire party and defeat the companion limit, because passenger benefit coverage was keyed on a client-supplied id that is not checked for uniqueness; a bag could be labelled “waived” while the traveller was being charged an overweight surcharge on it; the rate limiter ran *after* the database queries it existed to prevent; the 32 KB request cap could be walked past entirely; the airline FAQ invented a 62-inch size limit for any airline that publishes none; and the service worker cached error pages, so one 404 would hide the offline page for that URL until the next deploy.
- **Second production database change applied and verified** (migration 0003). `status` is now constrained at the database itself to the three documented values, so no hand-typed `UPDATE` can put an unrecognised value in. Rehearsed first — which caught two defects in my own draft, one of which would have emptied two tables. Afterwards: row counts unchanged, foreign keys intact, and a bad status value confirmed rejected by the live database.
- **The database and the dataset were reconciled.** Every id in production matches `data/reference-data.json` exactly, which is what item 4 below depends on and what makes the seed safe to re-run.

BLOCKERS Nothing is live until these are done

☐ 1. Find out what is actually serving the site — 10 min, do this first

Unresolved. The apex DNS record for `luggageliason.com` is a proxied CNAME to `packright-6z1.pages.dev`, but **no Pages project by that name exists** in your Cloudflare account (I checked all 10). Either it was deleted, renamed, or lives somewhere I cannot see. I could not load the site to check, because this environment blocks outbound web access.

Re-checked at 15:07 UTC on 29 August, after PR #2 merged: still 10 Pages projects, still none of them PackRight, and the apex CNAME still points at the missing one. Also still no `www` record.

This decides where the new frontend gets deployed, so resolve it before anything else. Open the Cloudflare dashboard → Workers & Pages and look for the project behind that hostname. If it is gone, create a new Pages project, then repoint the apex CNAME at its `*.pages.dev` subdomain and add `luggageliason.com` as a custom domain.

☐ 2. Deploy the API Worker — 5 min

The deployed Worker is still the pre-audit build from 31 July. The database is already migrated and is backward compatible, so the old Worker keeps serving until you replace it — there is no rush window and no downtime. I could not do this myself: `wrangler` is not authenticated in my environment.

```
# from the repo root
cd api
npx wrangler login
npm run deploy # runs typecheck + 40 tests, then deploys

# confirm it is live and reporting data freshness
curl -s https://packright-api.kevynsgrin.workers.dev/api/v1/health
```

☐ 3. Deploy the frontend — 10 min

Once item 1 tells you the project name. The build produces 19 prerendered HTML files plus a sitemap and service worker.

```
cd frontend
npm ci && npm run build
npx wrangler pages deploy dist --project-name=<PROJECT>
```

`public/_headers` and `public/_redirects` are **Cloudflare Pages specific**. Every security header and the one-year asset cache only take effect on Pages. On any other host they are inert files and the site ships with none of that protection.

TRUST Required before you drive traffic

☐ 4. Verify the airline dataset — the big one

Every record ships `status: "unverified"` with `verified_at: null`, so the site shows “Review pending” everywhere. That is deliberate and honest — the figures were carried over from the original build and have never been checked against the airlines. Use the two Gemini deep-research prompts I gave you; the second one ends with a JSON block shaped to drop straight into `data/reference-data.json`.

Southwest is the priority. It is modelled as two free checked bags, which changed during 2025. Anyone planning a Southwest trip on the current data gets a materially wrong answer.

Also worth capturing: real overweight and oversize fee *bands*. Four figures in the engine (\$100 oversize personal, \$100 gate carry-on, \$100 overweight, \$150 third bag) are inherited placeholders, shown with an “Assumption” badge. Real banded figures let those be deleted entirely.

One thing the research should settle: all eight airlines carry an identical 62-inch checked-bag linear limit. 62 inches is the industry-standard figure, so this may well be right — but eight identical unverified values is the signature of a single assumption copied across the table rather than eight figures read off eight airline pages. Worth confirming per airline rather than in aggregate. (The code no longer invents this number when it is missing, but it cannot tell you whether a present one was ever checked.)

Send the results back to me and I will update the dataset, flip verified records, regenerate the seed and re-run the tests.

☐ 5. Counsel review of the policy pages — external

`/privacy`, `/terms` and `/affiliate-disclosure` now name Oak and Main Developers LLC, give the Sacramento address, set California governing law and Sacramento County venue, and state how the site handles Do Not Track. Every factual claim was checked against the actual code — including correcting a claim that the app writes no browser storage, which stopped being true when the service worker landed.

They have still not been read by a lawyer. Worth an hour of someone’s time before you acquire traffic, particularly the liability limitation and the estimate disclaimer.

CONFIG Cloudflare settings — two done, the rest need a decision

☐ 6. Add `www`, or drop the redirect — 5 min

There is **no `www` DNS record at all** on the zone. The audit noticed `www` did not resolve; that is why. The redirect rule I shipped in `_redirects` can never fire until `www` exists.

Every other site in your portfolio (bountycharts, glp1-fund, ridereadyguide, benefitdial, gochujang, originespana, kbbqguide) has `www` as a Pages custom domain. This one is the odd one out.

I have **removed** the redirect rule, because it could never have worked: Cloudflare Pages `_redirects` matches on path only, so an absolute URL in the source position is inert, and the trailing `!` was Netlify syntax. A rule that looks like policy but does nothing is worse than none. To actually canonicalise `www`: add a proxied `www` CNAME to the Pages project, add `www.luggageliason.com` as a custom domain on it, then create a dashboard Redirect Rule (Rules → Redirect Rules) matching that hostname to the apex, 301, preserving path and query. Or decide `www` is simply not used and leave it absent.

☐ 7. Always Use HTTPS — done, nothing to do — 0 min

The zone setting `always_use_https` was **off**. I turned it **on** through the Cloudflare API and confirmed it read back as on. Nothing could break: the apex is not currently serving anything (see item 1), and the change only adds an HTTP → HTTPS redirect. Reversible at Dashboard → SSL/TLS → Edge Certificates → Always Use HTTPS if you ever want it off.

☐ 8. Cloudflare Web Analytics — verified, nothing to do — 0 min

I checked this against the Cloudflare API rather than leaving it to you. Web Analytics **is** enabled for `luggageliason.com` with auto-install (site tag `7e2f42ed...`), so the privacy page's claim is accurate. The beacon does not appear in the built output because Cloudflare injects it at the edge. Tick this and move on.

☐ 9. Decide on HSTS preload — 5 min, think first

`_headers` currently sends `max-age=63072000; includeSubDomains; preload`. The header is safe. *Submitting* to `hstspreload.org` is close to irreversible and forces HTTPS on every subdomain for years. Confirm every subdomain you will ever want is HTTPS-ready, then either submit at `hstspreload.org` or remove the `preload` token.

☐ 10. Durable rate limiting — 10 min

Update: the in-isolate limiter now rejects a request *before* it reaches the database, rather than after, and its 429 is no longer cacheable. That closes the wasteful part. It is still per-isolate and still not the authoritative control, so this item stands.

The Worker enforces a per-isolate limit and emits accurate `RateLimit-*` headers, but Workers isolates do not share memory, so it is best effort rather than an account-wide guarantee. Either enable the commented-out rate-limit binding in `api/wrangler.toml`, or add a WAF rate-limiting rule on `packright-api.kevynsgrin.workers.dev`. At current traffic this is not urgent.

SEARCH After the site is deployed

☐ 11. Google Search Console — 10 min

Good news: a `google-site-verification` TXT record is **already on the zone**, so the property is likely verified. Submit `https://luggageliason.com/sitemap.xml` (18 URLs), then use URL Inspection to request re-indexing of the home page — Google currently shows the old title *“frontend”* and that will not refresh on its own. Watch the Coverage report for soft-404s.

☐ 12. Bing Webmaster Tools — 10 min

Not verified as far as I can tell. You can import directly from Search Console once item 11 is done, which is the fastest route. Submit the same sitemap.

LATER Only when it applies

☐ 13. Affiliate programme

No partner links ship today, and the old placeholder cards are gone. `/affiliate-disclosure` sets the standard any future link must meet: a real approved destination, `rel="sponsored noopener norereferrer"`, and the disclosure next to the link rather than only on the policy page. Do not enable acquisition against unverified data — item 4 comes first.

☐ 14. Post-deploy smoke test — 3 min

Run this once the site is live. Every line should return what the comment says.

```
curl -sI https://luggageliason.com/ | grep -i "content-security-policy|strict-transport" #
both present
curl -s https://luggageliason.com/robots.txt | head -1 #
plain text, not HTML
curl -sI https://luggageliason.com/sitemap.xml | grep -i content-type #
application/xml
curl -so /dev/null -w "%{http_code}\n" https://luggageliason.com/no-such-page #
404, not 200
curl -s https://luggageliason.com/ | grep -o "<title>[^<]*" # NOT
"frontend"
```